Chapter 4

Hiding One Bit

4.1 Intro: Can we encrypt?

The Factoring Assumption implies that computing square roots modulo Blum integers N is hard: $[x^2 \bmod N]$ "hides" x, without knowing the factors of N.

Hiding information seems very useful for encryption! Suppose we had an encryption scheme such that the encryption of a message $m \in \mathbb{Z}_N^*$ was the ciphertext $c = [m^2 \bmod N]$, for $N = pq$. Is this encryption scheme workable under the Factoring Assumption?

There are two main problems.

- Correctness: How can we decrypt? Given the factors p, q of N, one can efficiently compute all four square roots of c. How is the decryptor supposed to figure out which root is the real plaintext message m?
- Security: The Factoring Assumption (by way of the reduction to Squaring) only guarantees that a PPT attacker **can't fully recover uniformly random messages** with non-negligible probability. Both restrictions undermine security. It doesn't prevent an attacker from:
  - fully recovering *non-random* messages, nor
  - *partially* recovering m (even for random m).

Hard problems must hide something, right? If FACTORING is hard, it seems like there must be *some* information about x that is hidden by $x^2 \bmod N$. Meaning, there some function $h(x)$ that it should be hard to compute given $(N, x^2 \bmod N)$ as input. Otherwise, one should be able to recover x completely, taking square roots and violating FACTORING. What information about x is hidden?[1]

In this chapter, we will see that indeed something is hidden about the square root: its least significant bit.

[1] Similar intuition seems like it should apply to any other hard problem. Later in the course we'll revisit this point, and see that there is a sense in which every hard problem must hide some information about its input.

4.2 Least significant bit and the principle root

The least-significant bit (LSB) of an integer is the same as its *parity*: 1 if it's odd, 0 if it's even. We apply this idea to the integers modulo N. Observe that two integers of different parity can be modulo an odd N. For example, $2 \equiv 7 \bmod 5$. So we define $\mathsf{LSB}(x)$ by looking at the canonical representative $[x \bmod N]$, which is unique.

Definition 4.1 (Least significant bit)**.** *For $x \in \mathbb{Z}_N$, we define its* least significant bit *as:*

$$\mathsf{LSB}(x) = \begin{cases} 0 & \text{if } [x \bmod N] \text{ is even} \\ 1 & \text{if } [x \bmod N] \text{ is odd.} \end{cases}$$

Example 4.1. Modulo $N = 5$: $\mathsf{LSB}(2) = \mathsf{LSB}(7) = 0$.

For reference we recall our sampler for Blum integers (slightly modified to return p and q along with $N = pq$ for later notational convenience). $\mathsf{Primes}(1^n)$ samples a uniformly random n-bit prime.

$\mathsf{Blum}(1^n)$

$p \leftarrow \mathsf{Primes}(1^n)$, $q \leftarrow \mathsf{Primes}(1^n)$ independently
while $(q = p)$ *OR* $(p \not\equiv 3 \bmod 4)$ *OR* $(q \not\equiv 3 \bmod 4)$ **do**
  $p \leftarrow \mathsf{Primes}(1^n)$, $q \leftarrow \mathsf{Primes}(1^n)$ independently
$N := pq$
return (p, q, N)

4.3 How to formalize "squaring hides LSB"

Let's try to formalize a claim that computing the LSB of the square root of y is hard. Our first attempts will be wrong, but instructive.

Theorem 4.1 (Attempt 1 (FALSE!))
If FACTORING*, then the following holds. For all PPT algorithms $\mathcal{A}$,*

$$\Pr\left[\widehat{b} = \mathsf{LSB}(x)\right] \text{ is negligible}$$

in the probability experiment

$$\begin{aligned} (p, q, N) &\leftarrow \mathsf{Blum}(1^n) \\ x &\xleftarrow{\$} \mathbb{Z}_N^* \\ y &:= [x^2 \bmod N] \\ \widehat{b} &\leftarrow \mathcal{A}(N, y). \end{aligned}$$

This is impossible, because $\mathsf{LSB}(x)$ only takes two possible values. The algorithm $\mathcal{A}$ that returns a random bit $\widehat{b} \xleftarrow{\$} \{0,1\}$ will satisfy[2]

$$\Pr\left[\widehat{b} = \mathsf{LSB}(x)\right] = \frac{1}{2}.$$

The way to fix this is by requiring that $\mathcal{A}$ can't guess $\mathsf{LSB}(x)$ with probability non-negligibly greater than 1/2:

Theorem 4.2 (Attempt 2 (TRIVIAL!))
If FACTORING, *then the following holds. For all PPT algorithms $\mathcal{A}$, there exists a negligible function* negl *such that for all sufficiently large n:*

$$\Pr\left[\widehat{b} = \mathsf{LSB}(x)\right] < \frac{1}{2} + \mathsf{negl}(n)$$

in the probability experiment

$$\begin{aligned}(p, q, N) &\leftarrow \mathsf{Blum}(1^n)\\ x &\xleftarrow{\$} \mathbb{Z}_N^*\\ y &:= [x^2 \bmod N]\\ \widehat{b} &\leftarrow \mathcal{A}(N, y).\end{aligned}$$

This version of the theorem isn't impossible...it's trivial! The problem is that y contains no information about $\mathsf{LSB}(x)$, by the following claim. Put another way: "the LSB of the square root of y mod N" is not well-defined, because "the square root of y mod N" is not well defined.

Claim 4.1. *Let $N = pq$ be the product of distinct odd primes, and let $y \in QR_N$ be a quadratic residue. Exactly half of the (four) square roots of y have LSB equal to 0.*

Proof. If x is a square root of y, then so is $-x = N - x$. Moreover, x and $-x$ are distinct: $-x \not\equiv x \bmod N$. Because $x + (N - x) = N$ and N is odd:

$$\mathsf{LSB}(x) \neq \mathsf{LSB}(-x).$$

By the above, for every root x of y with $\mathsf{LSB}(x) = 0$, there is another root $-x$ with $\mathsf{LSB}(-x) = 1$. Therefore, exactly half the roots have LSB equal to 0. □

4.3.1 Principle square root modulo Blum integers

To solve the problem above, we need to disambiguate "the square root of y modulo N." We want to say one of the four roots of y is the *principle root*, and the other three roots are

[2] In fact, $\mathcal{A}$ doesn't even need to randomize. This is because exactly half the elements $x \in \mathbb{Z}_N^*$ have $\mathsf{LSB}(x) = 0$. Hence an algorithm $\mathcal{A}$ that ignores its input and always returns $\widehat{b} = 0$ it also achieve $\Pr\left[\widehat{b} = \mathsf{LSB}(x)\right] = \frac{1}{2}$. Still, guessing randomly is often a good strategy for designing an adversary / reduction with a simple analysis.

secondary roots. This is analogous to how we define the principle root of a real number to be its positive square root: $\sqrt{4} = 2$ even though $(-2)^2 = 4$.

When $N = pq$ is a Blum integer (i.e., $p, q \equiv 3 \bmod 4$), there is a crisp way to distinguish one of the roots from the other three. In fact, you proved this in one of the exercises from the previous chapter!

Lemma 4.1. *Let N be a Blum integer, $y \in QR_N$. Exactly one square root of y is a quadratic residue.*

Proof. Let $p \equiv 3 \bmod 4$ be a prime. First, we show that for every $ax \in Z_p^*$, exactly one of $\pm x$ is a quadratic residue modulo p. Recall Euler's Criterion: $a \in QR_p$ if and only if $a^{(p-1)/2} \equiv 1 \bmod p$. Applying Euler's Criterion to $-x$, we get

$$(-x)^{(p-1)/2} \equiv -(1)^{(p-1)/2} \cdot x^{(p-1)/2} \equiv (-1)^{(4k+3-1)/2} \cdot x^{(p-1)/2} \equiv (-1) \cdot x^{(p-1)/2} \bmod p.$$

Thus exactly one of x and $-x$ is in QR_p.

Let $N = pq$ for $p, q \equiv 3 \bmod 4$ be a Blum integer. Let $y \in QR_N$. There exist exactly 4 roots of y, and they are $\mathrm{crt}(\pm a, \pm b)$ for some values $a \in \mathbb{Z}_p^*$, $b \in Z_q^*$. By the previous argument, exactly one of $\pm a$ is in QR_p and exactly one of $\pm b$ is in QR_q. Hence, there is exactly one root for which both its CRT components are themselves quadratic residues (modulo the respective primes).

An element $x = \mathrm{crt}(u, v) \in \mathbb{Z}_N^*$ is in QR_N if and only if both $u \in QR_p$ and $v \in QR_q$. Therefore, exactly one root of y is in QR_N. □

4.3.2 Squaring hides LSB

The only change we make from the previous attempt is to sample x uniformly among the *principle roots* of elements of QR_N. That is, $x \xleftarrow{\$} QR_N$ rather than $x \xleftarrow{\$} \mathbb{Z}_N^*$.

Theorem 4.3

If FACTORING*, then the following holds. For all PPT algorithms $\mathcal{A}$, there exists a negligible function* negl *such that for all sufficiently large n:*

$$\Pr\left[\widehat{b} = \mathsf{LSB}(x)\right] < \frac{1}{2} + \mathsf{negl}(n)$$

in the probability experiment

$$\begin{aligned}(p, q, N) &\leftarrow \mathsf{Blum}(1^n)\\ x &\xleftarrow{\$} QR_N\\ y &:= [x^2 \bmod N]\\ \widehat{b} &\leftarrow \mathcal{A}(N, y).\end{aligned}$$

The proof of this theorem is complex and way beyond the scope of this class. We will prove the following weaker lemma, which is one of the main steps of the proof. The lemma states that $\mathcal{A}$ can't **perfectly** recover the LSB, for moduli N where $2 \in QR_N$. By perfect,

we mean that $\mathcal{A}(N, y)$ always correctly returns $\mathsf{LSB}(x)$ for the principle root x of y. The full theorem requires extending it too all moduli, and strengthening the 1 to $1/2 + \mathsf{negl}(n)$.

Lemma 4.2 (Reduction from square root to perfect LSB when 2 is a quadratic residue)**.** *Suppose there exists a PPT $\mathcal{A}$ such that for all Blum integers N the following holds. If $2 \in QR_N$, then*

$$\Pr\left[\widehat{b} = \mathsf{LSB}(x)\right] = 1$$

in the probability experiment $x \xleftarrow{\$} QR_N$, $y := [x^2 \bmod N]$, $\widehat{b} \leftarrow \mathcal{A}(N, y)$.

Then there exists a PPT $\mathcal{B}$ such that for all Blum integers N the following holds. If $2 \in QR_N$, then

$$\Pr\left[\widehat{x}^2 \equiv y \bmod N\right] = 1$$

in the probability experiment $x \xleftarrow{\$} QR_N$, $y := [x^2 \bmod N]$, $\widehat{x} \leftarrow \mathcal{B}(N, y)$.

Proof. Let N be a Blum integer such that $2 \in QR_N$, and let $y = [x^2 \bmod N]$ for some $x \in QR_N$.

Our algorithm $\mathcal{B}(N, y)$ will find x in two stages. In Stage 1, it will use $\mathcal{A}$ to compute the first k bits $b_1, \ldots, b_k$ of the binary representation of x/N:

$$\frac{x}{N} = 0.b_1 b_2 b_3 \ldots.$$

In Stage 2, it will use the previous result to compute x.

$\mathcal{B}(N, y)$

$k = \lceil \log_2 N \rceil + 1$

// Stage 1
for $i = 1, \ldots, k$ **do**
  $\widehat{b}_i := \mathcal{A}([4^i y \bmod N])$

// Stage 2
$\widehat{x}_k := N \cdot 0.\widehat{b}_1 \widehat{b}_2 \ldots \widehat{b}_k$
$\widehat{x} = \mathsf{Round}(\widehat{x}_k)$
return $\widehat{x}$

$\mathcal{B}$ is PPT: it executes poly-many operations (k is polynomial in the length of $\mathcal{B}$'s input), each of which is computable in PPT (by assumption on $\mathcal{A}$).

For correctness, we must show two things: (1) $\widehat{b}_i = b_i$ for all i; and, if so, (2) $\widehat{x} = x$.

(1) Observe that $2^i x$ is a square root of $4^i y$ modulo N. Because $2 \in QR_N$ and $x \in QR_N$, we have that $2^i x \in QR_N$. Because N is a Blum integer, $2^i x$ is the unique root of $[4^i y \bmod N]$ that is itself a quadratic residue. By assumption on $\mathcal{A}$:

$$\widehat{b}_i = \mathcal{A}([4^i y \bmod N]) = \mathsf{LSB}([2^i x \bmod N]).$$

It remains to show that $\mathsf{LSB}([2^i x \bmod N]) = b_i$, the ith bit of the binary expansion of x/N. By definition,[3] b_i is the least significant bit (over the integers) of $\lfloor 2^i(x/N) \rfloor$. It encodes whether this quantity is even or odd.

By definition of the modular reduction operation:

$$2^i x = N \left\lfloor \frac{2^i x}{N} \right\rfloor + [2^i x \bmod N].$$

The left hand side is even and N is odd. Therefore,

$$\left\lfloor \frac{2^i x}{N} \right\rfloor \text{ is odd } \iff [2^i x \bmod N] \text{ is odd.}$$

Equivalently:

$$b_i = \mathsf{LSB}([2^i x \bmod N]).$$

(2) For $\mathsf{Round}(\widehat{x}_k) = x$, it suffices that $|\widehat{x}_k - x| < 1/2$.

$$\begin{aligned} |\widehat{x}_k - x| &= |N(0.b_1 b_2 \cdots b_k \ - \ 0.b_1 b_2 \cdots b_k b_{k+1} b_{k+2} \cdots)| \\ &= N \cdot 0.\underbrace{00\cdots0}_{k} b_{k+1} b_{k+2} \cdots \\ &< N \cdot 2^{-k} \\ &< \frac{1}{2} \end{aligned}$$

□

4.4 Outro: We can encrypt!

We've discovered that the hardness of Factoring implies that for quadratic residues x, the squaring function $[x^2 \bmod N]$ "hides" one bit of information about x: whether it is odd or even.

Hiding information seems very useful for encryption! Next chapter, we're going to use this to construct a **public key encryption scheme** for one bit messages. The public key (used to encrypt) is a Blum integer N. The secret key (used to decrypt) is the factorization (p, q). A message $m \in \{0, 1\}$ is a bit. The encryption algorithm is as follows:

$\mathsf{Enc}_N(m)$

$x \xleftarrow{\$} QR_N$
$y := [x^2 \bmod N]$
$c := (\mathsf{LSB}(x) \oplus m;\ y)$
return c

Let's see how this approach solves the problems that we ran into with our first attempt at the beginning of these notes.

[3] Multiplying by 2^i moves the decimal i places to the right: $b_1 \cdots b_i.b_{i+1} \cdots$. Taking the floor removes the fractional part.

- Correctness: By sampling x from the set of quadratic residues and requiring N to be a Blum integer, x is the principle root of y. It can be uniquely recovered given the prime factors of N.
- Security: No PPT algorithm can guess $\mathsf{LSB}(x)$ better than random chance, even given (N, y). We will use this to show that no PPT adversary can guess m better than random chance. Though it's not at all obvious, this implies that no partial information about m is revealed regardless of the distribution of m.

4.5 Exercise

Exercise 4.1 (One-sided and two-sided prediction bounds)**.** Fix a randomized polynomial-time algorithm Samp. On input 1^n, it outputs a pair (z, b), where $b \in \{0, 1\}$.

A **predictor** $\mathcal{A}$ receives z and tries to guess b. For a predictor $\mathcal{A}$, define

$$\mathbf{Adv}^{\mathsf{pred}}_{\mathcal{A}}(n) := \Pr_{(z,b)\leftarrow\mathsf{Samp}(1^n)}\left[\mathcal{A}(1^n, z) = b\right]$$

The probability is over the randomness of both Samp and $\mathcal{A}$.

Prove that the following statements are equivalent:

- For every PPT predictor $\mathcal{A}$ whose output is always in $\{0, 1\}$, there exists a negligible function negl such that, for all sufficiently large n,

$$\mathbf{Adv}^{\mathsf{pred}}_{\mathcal{A}}(n) < \frac{1}{2} + \mathsf{negl}(n).$$

- For every PPT predictor $\mathcal{A}$, the function

$$\left|\mathbf{Adv}^{\mathsf{pred}}_{\mathcal{A}}(n) - \frac{1}{2}\right|$$

  is negligible.

Note on the correction: Without requiring $\mathcal{A}$'s output to be in the set $\{0, 1\}$, the statement is false. For a bonus exercise: give a counter-example.

Exercise 4.2. Sometimes changing a sampling domain is innocuous, and sometimes it's not. This problem refers to Theorem 3.3 from the previous notes and its proof.

(a) Let $\pi : \mathbb{Z} \to \{\mathsf{TRUE}, \mathsf{FALSE}\}$ be some property over the integers (e.g., even, multiple of 7). Prove the following:

$$\Pr_{\substack{N\leftarrow\mathsf{Blum}(1^n)\\ x \xleftarrow{\$} \mathbb{Z}_N^*}}[\pi(x)] \text{ is negligible} \quad \Longrightarrow \quad \Pr_{\substack{N\leftarrow\mathsf{Blum}(1^n)\\ x \xleftarrow{\$} QR_N}}[\pi(x)] \text{ is negligible.}$$

(b) Prove the following **by reduction**. Observe that claim is the same as Theorem 3.3 except for how x is sampled. You may use Theorem 3.3 without reproving it.

Hint: The previous part is a hint for this part. Suppose for contradiction that the claim is false, and show how to violate Theorem 3.3. Don't emulate the proof of Theorem 3.3

— the solution is much simpler.
Clarification: You can't simply apply part (a) out of the box. Meaning, it is not simple to define a property π with respect to which applying (a) proves the statement in (b).

Claim 4.2. *If* FACTORING*, then the following holds. For all PPT algorithms $\mathcal{A}$,*

$$\Pr\left[\hat{x}^2 \equiv y \bmod N\right] \text{ is negligible,}$$

in the probability experiment $(p, q, N) \leftarrow \mathsf{Blum}(1^n)$, $x \xleftarrow{\$} QR_N$, $y := [x^2 \bmod N]$, $\hat{x} \leftarrow \mathcal{A}(N, y)$.

(c) Suppose that we changed the reduction from the proof of Theorem 3.3. Again, the only difference is how we're sampling x. (This time, we're changing how the reduction samples x, not the theorem statement.)

$\mathcal{B}'(N)$

$x \xleftarrow{\$} QR_N$; $y \leftarrow \mathsf{sqr}_N(x)$
$\hat{x} \leftarrow \mathcal{A}(N, y)$ **if** $\hat{x}^2 \not\equiv y \bmod N$ **then return** $\perp$
if $\hat{x} \equiv \pm x \bmod N$ **then return** $\perp$
$\hat{p} \leftarrow \mathsf{gcd}(x + \hat{x}, N)$
return $\hat{p}$

Suppose we replaced $\mathcal{B}$ with $\mathcal{B}'$ in the proof of Theorem 3.3, leaving the rest of the proof as is. Explain why the proof is no longer correct. (Find the sentence(s) that are no longer true!)

Exercise 4.3 (Randomized Self-Reduction)**.** This problem shows that finding the square root of an *arbitrary* quadratic residue is no harder than finding the square root of a *random* quadratic residue.

For every $N \geq 2$, define the **minimal quadratic residue** for N as the quadratic residue with the smallest representative in $\mathbb{Z}_N$:

$$y^* := \arg\min_{y \in QR_N} [y \bmod N].$$

Suppose $\mathcal{A}$ is a PPT principle-root finding algorithm such that:

$$\Pr[(\hat{x}^2 \equiv y \bmod N) \wedge (\hat{x} \in QR_N)] = \epsilon(n)$$

in the probability experiment

$$\begin{aligned} N &\leftarrow \mathsf{Blum}(1^n) \\ y &\xleftarrow{\$} QR_N \\ \hat{x} &\leftarrow \mathcal{A}(N, y). \end{aligned}$$

Construct a PPT algorithm $\mathcal{B}$ such that:

$$\Pr[(\hat{x}^2 \equiv y \bmod N) \wedge (\hat{x} \in QR_N)] = \epsilon(n)$$

in the probability experiment

$$\begin{aligned} N &\leftarrow \mathsf{Blum}(1^n) \\ y^* &:= \text{the minimal quadratic residue for } N \\ \hat{x} &\leftarrow \mathcal{B}(N, y^*). \end{aligned}$$

Justify your answer.

Hint: This problem doesn't depend on any special property of the y^* defined above. The only thing I'm using is that for every N, there is one quadratic residue y^* that is distinguished from all others. The solution to the problem would be exactly the same if we replaced the minimum with the maximum (or: the median, or the 15th from the bottom, or the one with the most 1s in its binary representation, or any other way of distinguishing a specific residue).